COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	12/9/2026	**DATE SUBMITTED:**	15/9/2026
NAME:	Duenas Ralph Florenz F.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1

1. PROCEDURE OUTPUT

Table 8-1.	Array of Values for Sort Algorithm Testing
Code + Console Screenshot	
Observations	The code makes an array of 100 numbers and fills it with random values between 0 and 99 using rand(). It then uses a function to print all these numbers out with spaces in between them. Looking at the output, you can see a mixed-up list of 100 numbers, which shows that the random data set was made correctly.

```cpp
#include <iostream>
#include "sortingAlgo.h"
#include <cstdlib>
#include <ctime>

const int maxSize = 100;

void displayArray(int arr[]);

int main() {
    int array[maxSize];

    std::srand(std::time(0));

    for (int i = 0; i < maxSize; i++){
        array[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArray(array);

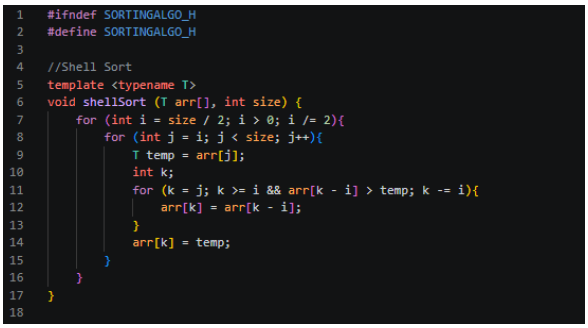
    /*
    std::cout << "\nShell Sort:\n";
    shellSort(array, maxSize);
    displayArray(array);
    */

    /*
    std::cout << "\nMerge Sort:\n";
    mergeSort(array, 0, maxSize - 1);
    displayArray(array);
    */

    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
    displayArray(array);

}

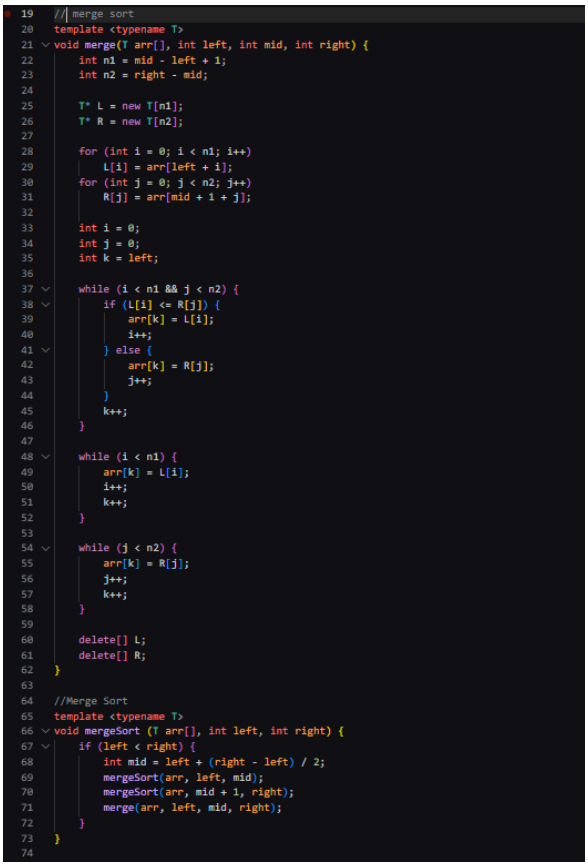
void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Table 8-2.	Shell Sort Technique
Code + Console Screenshot	
Observations	The code uses a shellSort() function saved in a header file to sort the random numbers. It compares elements separated by a gap distance and gradually shrinks the gap size down to 1 to sort smaller sub-lists. The console output shows all 100 numbers lined up from lowest to highest, showing that the shell sort worked.

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

//Shell Sort
template <typename T>
void shellSort (T arr[], int size) {
    for (int i = size / 2; i > 0; i /= 2){
        for (int j = i; j < size; j++){
            T temp = arr[j];
            int k;
            for (k = j; k >= i && arr[k - i] > temp; k -= i){
                arr[k] = arr[k - i];
            }
            arr[k] = temp;
        }
    }
}

```

Table 8-3.	Merge Sort Algorithm
Code + Console Screenshot	
Observations	The code uses a mergeSort() function saved in a header file to sort the random numbers. It repeatedly splits the array into two equal halves until single elements remain, then merges the sorted sub-arrays back together using dynamic arrays. The console output shows the list fully ordered from smallest to largest, proving that the divide-and-conquer merge algorithm worked properly.

```
// merge sort
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

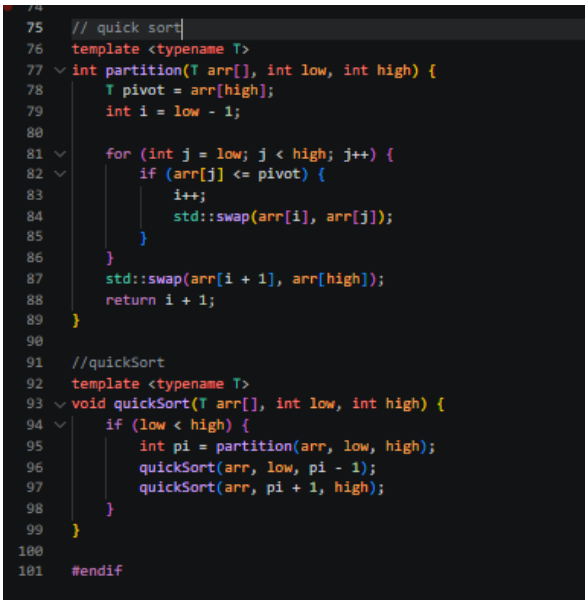
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

//Merge Sort
template <typename T>
void mergeSort (T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Table 8-4.	Quick Sort Algorithm

Code + Console Screenshot	
Observations	The code uses a quickSort() function saved in a header file to sort the random numbers. It chooses a pivot value and rearranges elements so smaller items move to the left and larger items move to the right, then recursively sorts both sides. The console output shows the numbers neatly organized from 0 to 99, showing that partitioning and sorting the elements worked as intended.

```
// quick sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

//quickSort
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

#endif
```

OUTPUT

```
Unsorted Array:
10 79 47 5 35 33 11 70 71 16 25 62 33 87 37 37 85 42 82 79 57 66 46 24 69 17 36 30 25 11 86 76 37 14 4 59 31 35 31 34 61 22 80 70 37 52 67 85 97
48 62 20 42 61 50 96 18 18 17 98 36 4 28 62 67 64 61 72 8 67 48 91 9 81 1 36 55 11 66 11 83 24 18 12 42 93 38 16 52 85 99 88 6 5 84 39 40 79 45

Shell Sort:
1 3 3 4 8 9 11 11 16 16 17 18 21 21 22 23 24 24 26 27 28 28 29 29 30 31 32 33 34 34 34 35 35 35 36 36 37 37 37 37 39 41 42 43 43 44 45 46 46 48
8 49 50 50 51 51 52 52 53 53 55 56 56 59 59 61 62 62 64 65 65 67 68 68 68 69 71 71 71 72 74 76 77 77 78 78 80 80 81 82 83 84 88 89 90 91 92 93 9

Merge Sort:
0 1 4 5 5 7 9 11 11 12 12 12 13 16 17 17 19 19 20 20 21 21 23 23 25 26 26 28 30 33 33 34 34 35 37 37 37 37 38 39 40 44 47 48 49 49 49 51 51 52 5
 54 56 56 58 61 61 62 62 64 64 65 66 66 68 72 73 75 75 75 76 77 77 77 77 79 79 80 80 80 82 82 85 85 86 88 88 89 89 91 92 92 93 93 94 95 96 97 99

Quick Sort:
0 0 0 1 3 5 6 9 12 13 13 15 15 15 21 21 22 24 25 26 28 28 29 29 31 32 32 34 35 37 37 37 38 39 39 40 41 42 44 45 47 47 48 50 51 51 51 52 54 56 56
59 60 60 60 60 61 62 63 63 69 69 71 72 73 73 76 76 76 76 77 77 77 78 78 79 79 79 81 82 83 85 85 85 86 86 88 88 89 91 93 94 95 95 96 97 98 98 99
PS C:\Users\chery\OneDrive\Documents\HOA8>
```

SUPPLEMENTARY ACTIVITY

Prob1.cpp

hoa8supp prob1 > prob.cpp > main()

```cpp
#include <iostream>
#include "prob1.h"

int main() {
    int arr[] = {34, 12, 25, 9, 19};
    int n = 5;

    std::cout << "Unsorted Array:\n";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n\n";

    hybridQuickSort(arr, 0, n - 1);

    std::cout << "Sorted Array:\n";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Prob.1.h

```h
#ifndef PROB1_H
#define PROB1_H

// Partition function from Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Simple Selection Sort for sub-lists
template <typename T>
void selectionSort(T arr[], int start, int end) {
    for (int i = start; i < end; i++) {
        int minIdx = i;
        for (int j = i + 1; j <= end; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        T temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

// Partitions first, then sorts left and right sub-lists
template <typename T>
void hybridQuickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Sort sub-lists using selection sort
        selectionSort(arr, low, pi - 1);
        selectionSort(arr, pi + 1, high);
    }
}

#endif
```

OUTPUT

```
Sorted Array:
9 12 19 25 34
```

Prob2.cpp

```cpp
#include <iostream>
#include "Prob2.h"

// Forward declaration matching the displayArray definition below
void displayArr(int arr[], int size);

int main() {
    // create the array in the problem
    int arr[15] = {4, 34, 29, 48, 53, 87, 12, 30,
                   44, 25, 93, 67, 43, 19, 74};

    // get the number of elements
    int n = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original Array:\n";
    displayArr(arr, n);

    // perform quick sort
    quickSort(arr, 0, n - 1);

    std::cout << "\nSorted Array:\n";
    displayArr(arr, n);

    return 0;
}

void displayArr(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Prob2.h

```cpp
#ifndef PROB2_H
#define PROB2_H

// Helper partition function for Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;


    // scans every number in the array for swapping
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// quickSort
template <typename T>
void quickSort(T arr[], int low, int high) {

    //checker if low < high
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

#endif
```

OUTPUT

```
Original Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

Sorted Array:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

- **Summary of lessons learned**

In this HOA, I learned how to use and compare three advanced sorting methods, which are Shell Sort, Merge Sort, and Quick Sort, to arrange numbers from smallest to largest. I learned that Shell Sort works by comparing numbers separated by a gap that gets smaller over time, Merge Sort splits the list into smaller halves and merges them back together in order, and Quick Sort selects a pivot value to separate smaller and larger numbers. I also figured out how to write reusable template functions in separate header files using C++. Finally, I discovered how to mix two algorithms together, like combining Quick Sort's partitioning with Selection Sort, to create a hybrid sorting function.

- **Analysis of the supplementary activity**

In the supplementary activity, the code demonstrates two separate problems using advanced sorting ideas. In Problem 1, Prob.1.h creates a hybrid sorting function that uses Quick Sort's partition() method to divide the numbers around a pivot and then applies selectionSort() on the resulting sub-lists to produce a sorted output. In Problem 2, Prob2.cpp creates an array of 15 unsorted numbers and passes it to Prob2.h, which uses a full quickSort() algorithm with partition() to divide and rearrange the items recursively until the entire array is displayed in increasing order on the console.

- **Concluding statement / Feedback:**

I think I did very well in this HOA. I was able to successfully make the code work across separate main files and header files, run all the sorting algorithms correctly, and display the expected outputs. However, I still need more practice with picking the best pivot element in Quick Sort and understanding how dynamic memory allocation works inside recursive sorting functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concept and the flow of the code itself.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Copilot to elaborate the thoughts I have in my mind without replacing the**

COMPUTER ENGINEERING

QUEZON CITY

original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.